Connect4 OO Design Report

CM500288 Principles of Programming — Graded Assignment 2

Made by: Juan David Arguello Plata

1. Introduction

Connect4 is a game for two players with a board/grid of 6x7. Each player is expected to deposit a colorful token into the grid and the game ends when a player connects 4 tokens (with their chosen color) within the board, either horizontally, vertically or through a diagonal ("Connect Four", n.d.).

An implementation of this game was presented (MyConnectFour.java). However, multiple errors were found during its inspection. Principally syntax and logical errors. Regarding its structure, it is conceived as a single-class program in which all its actors, including the game board, player's input and win detection system, coexist and are defined on the same functions/methods. It is highly illegible and hard to understand, a given characteristic of a poorly designed algorithm.

In this report, an explanation of the four key object-oriented principles (modularisation, encapsulation, inheritance and abstraction) is presented and explained to correct and redesign the given implementation.

2. Current design

The problem with the current design is that it lacks order and logic. It was created without a plan and simply looking to test direct logic. It did not implement any Object-Oriented Principle nor good practices. This single-class design violates the *Single Responsibility* principle stated (Martin, 2017) because the board state, I/O handling and win detection system works in the same place.

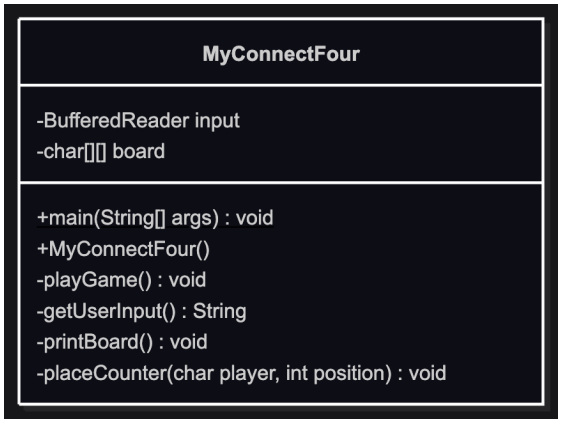

Figure 1. Current design. *Made with Mermaid*.

As seen on Figure 1, current design consists of just one class with several methods. It has two attributes: *input* and *board*. Methods like **playGame** and **placeCounter** are big and hard to understand. The first has nearly 100 lines, while the other is around 30 lines.

This implementation had several errors, but can be classified as:

- Syntax errors: lots of missing semi-colons, class-name did not match the constructor's name, and misspelled variables (Downey, 2019) were just a few examples of what was found in this file. Once corrected these errors, the logical ones popped up instantly.
- Logical errors: players were given their colors, red and yellow. It wasn't clear who was who. But when you interact with the command line, following game's instructions, it simply didn't turn out well, as seen on Figure 2. Basically, it is unplayable. The tokens are not positioned correctly and when it says that there is a winner, it does not stop and let the players continue with their match.

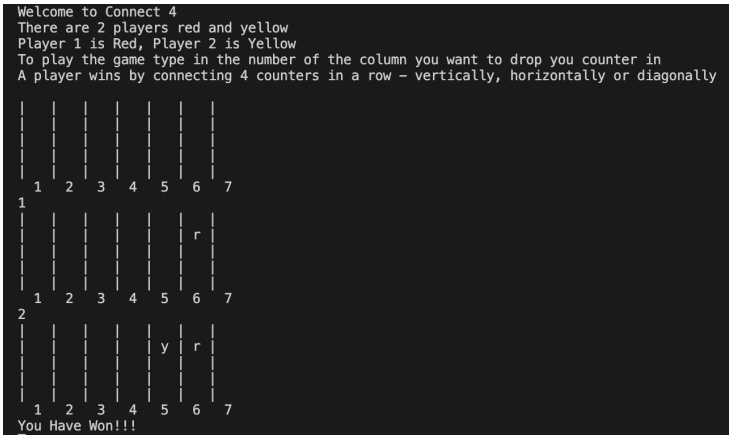

Figure 2. Logical errors.

As seen on Figure 2, there were two inputs and it did not work as expected.

3. New Design

There are several actors that correlate between each other. There are the concepts of 'Player', 'Board', 'Gameplay' and 'Rules' that could benefit from the perspective of using Object-Oriented principles. A redesign proposal can be appreciated on Figure 3.

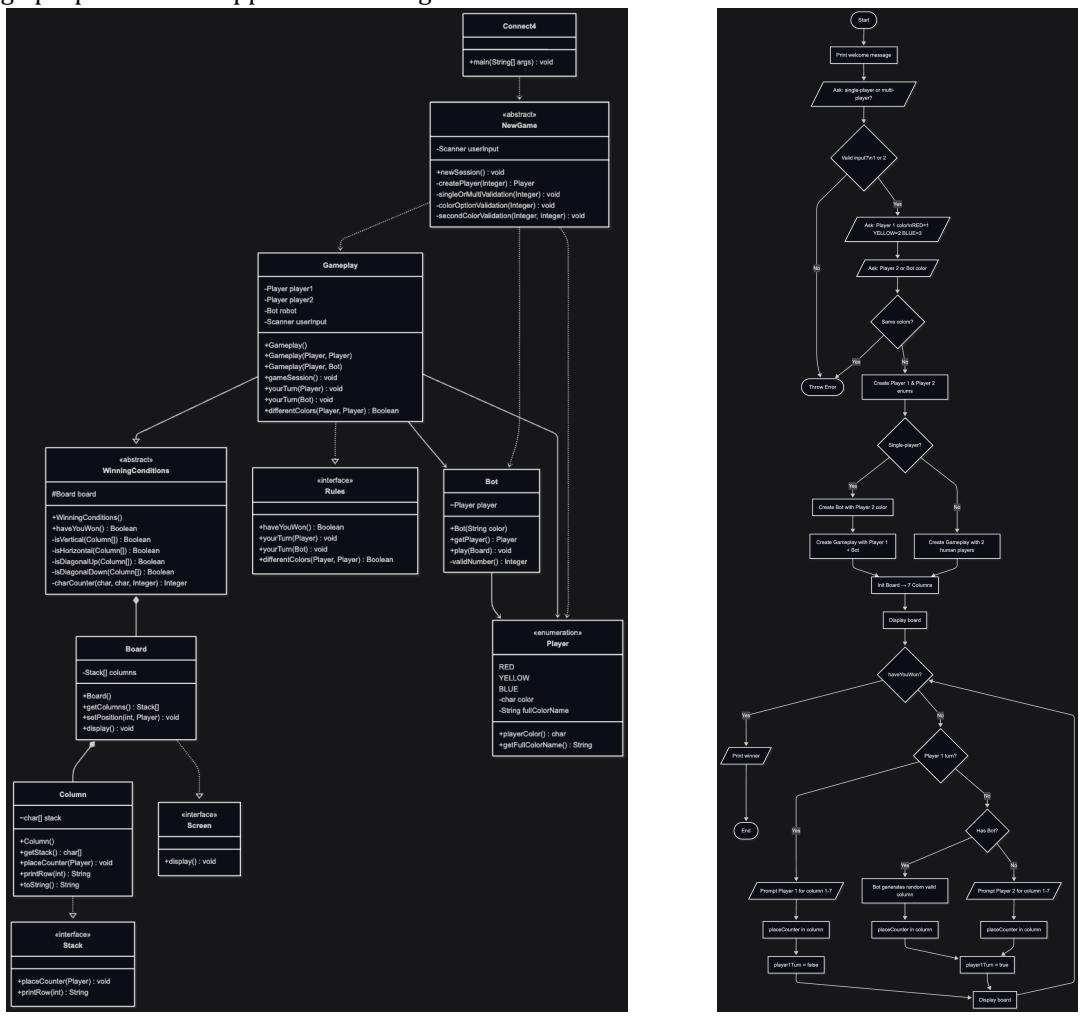

Figure 3. Redesign of Connect4. Class (left) & Flowchart (right) diagrams. *Made with Mermaid*.

3.1. Game's logic

As seen on Figure 3 (right), the flow starts with the abstract concept of 'NewGame', whose objective is just to define the configuration of the match. It asks the player whether the session will be single or multiplayer. It also asks the user for the colors of his/her player and its opponent. Finally, it uses this information to create the 'Player' and 'Bot' objects, if single player was selected.

Then, a 'Gameplay' object is created. As shown on Figure 3 (left), everything is orchestrated around this concept. It has the 'Player' objects, whether human or 'Bot' players; it has 'Rules' that defines whether a player has won the match or how to behave when there's a human or a bot's turn. It inherits the abstract concept of 'WinningConditions', starting a new 'Board' of the game and constantly evaluating the logic of vertical, horizontal and diagonal considerations to conclude whether a player has won the match or not.

The 'Board' object is made by 7 'Column' objects. Each 'Column' receives the token from the player and store it in a 'stack' variable, positioning it one after the other. Every turn, the 'Board' is printed to see the status of the match and let each player decide their next move.

3.2. Object-Oriented Design

As explained in section 3.1, each object has its place and its correlation between each other is what makes this a solid OO-Design based on *modularization*, *inheritance* and *abstractions*, as shown on Figure 3.

The code is split by **responsibility**, not just by class:

- The models package owns **data** — what the game is made of (Board, Column, Player, Bot).
- The services package owns **logic** — what the game does (NewGame sets up the session, WinningConditions detects wins, Gameplay runs the loop).

This means each piece can be read, tested, or changed in isolation (Martin, 2017). For example, you could swap out the win-detection algorithm in 'WinningConditions' without touching the 'Gameplay' or any other model.

On the other hand, there are several abstractions that helps to understand *what* an object can do, but not necessarily *how* it does it (Downey, 2019). For instance, the 'Rules' interface declares the behavioral contract for any game session and uses polymorphism through the methods 'haveYouWon', 'yourTurn(Player)' and 'yourTurn(Bot)' to dynamically set the behavior of each match.

Abstraction was also used at the definition of the 'WinningConditions' class, which contains all possible ways for a player to win a match (either horizontal, vertical or diagonal). It was defined as an abstract class to contain this logic and to be inherited at each 'Gameplay'. This logic is used in each 'gameSession' to define whether a player has won the match or not.

This design was strongly structured based on the SOLID principles (Martin, 2017). Table 1 explains how this redesign fulfills each principle.

Principle	Description	How the Connect4 redesign fulfils it
Single Responsibility (SRP)	Every class should have one, and only one, reason to change (Martin, 2017).	Each class owns a single concern: 'WinningConditions' detects wins, 'Board' manages the grid, 'NewGame' configures the session, and 'Gameplay' runs the loop. A change to win-detection logic touches only 'WinningConditions' and propagates nowhere else.
Open/Closed (OCP)	A class should be open for extension but closed for modification (Martin, 2017).	The 'Rules' interface allows new game modes to be introduced by implementing the interface, without modifying 'Gameplay'. Similarly, adding a new player colour to the 'Player' enumeration extends the system without altering any existing game logic.
Liskov Substitution (LSP)	Objects of a derived type must be substitutable for their base type without altering the correctness of the program (Martin, 2017).	Gameplay' interacts with both human and bot turns through the Rules interface. Either can be substituted into the game loop without breaking its behaviour, as both honour the same behavioural contract.
Interface Segregation (ISP)	No class should be forced to implement methods it does not use; interfaces should be small and focused (Martin, 2017).	Two focused interfaces are defined: 'Rules' governs game-session behaviour, while 'Screen' governs display behaviour. 'Column' implements only 'Screen'. Iit is never burdened with game-logic methods it has no use for.
Dependency Inversion (DIP)	High-level modules should not depend on low-level modules; both should depend on abstractions (Martin, 2017).	Gameplay' depends on the 'Rules' interface and inherits from the abstract 'WinningConditions' class — never on concrete implementations such as 'Bot' or a specific win-checking algorithm. Concrete details are confined to the lower-level classes, leaving the orchestration layer stable and independent.

Table 1. SOLID principles.

References

1. Connect Four. (n.d.). In *Wikipedia*. Retrieved June 1, 2026, from https://en.wikipedia.org/wiki/Connect_Four
2. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
3. Downey, A. B., & Mayfield, C. (2019). *Think Java: How to think like a computer scientist* (2nd ed.). O'Reilly Media.